Blocked Dynamic Sparse Attention

Mehul Goel (mehulg), Saransh Agrawal (saransh2)
Website: https://mehulgoel873.github.io/15418-final-project/

Summary

Background

Attention

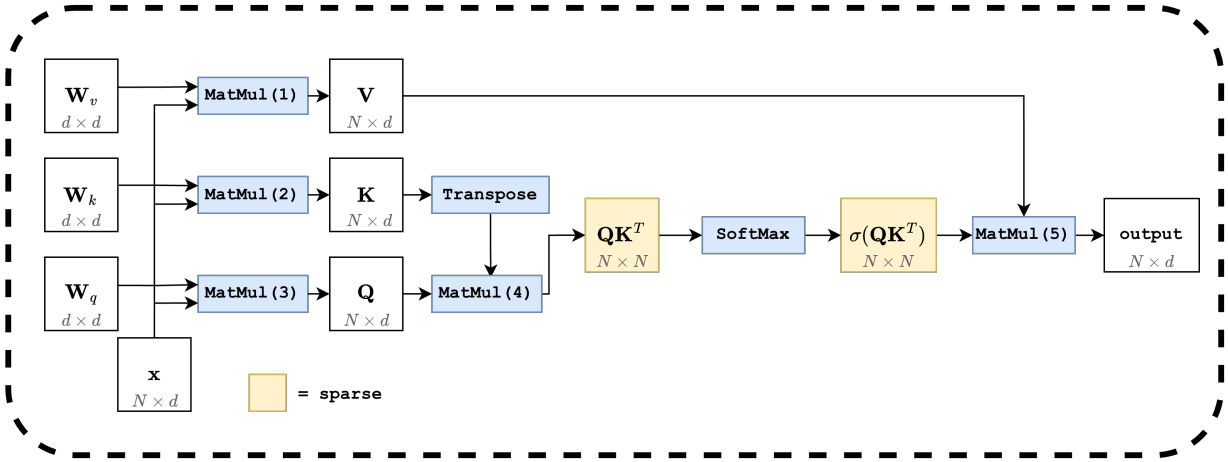


Figure 1: The attention block. Three projections (MatMuls (1)–(3)) produce Q, K, V; MatMul (4) and the softmax operate on an $N \times N$ matrix (highlighted), and MatMul (5) collapses back to $N \times d$.

Modern transformers process a sequence of tokens by repeatedly applying an *attention* operation. Each token is represented as a d-dimensional embedding vector, so the input to an attention layer is a matrix $\mathbf{x} \in \mathbb{R}^{N \times d}$, where N is the number of tokens (the *context length*) and d is the embedding dimension.

Figure 1 sketches the full attention block. The input $\mathbf{x}$ is first projected by three learned $d \times d$ weight matrices $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ (MatMuls (1)–(3)) into a query Q, a key K, and a value V, each of shape $N \times d$. The layer then computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V.$$

MatMul (4) forms the $N \times N$ score matrix QK^T, a row-wise softmax turns these scores into normalized weights, and MatMul (5) multiplies them by V.

The two highlighted stages in Figure 1 scale with the *square* of the sequence length. MatMul (4) materializes an $N \times N$ matrix, the softmax sweeps over it row by row, and MatMul (5) reads it

again. In practical settings $N \gg d$. Today's frontier models target context lengths of a million tokens or more, while d is typically only 64 or 128 per head, so a single attention layer would touch on the order of 10^{12} entries in its intermediate matrix. Both the arithmetic and the memory traffic of storing and revisiting an $N \times N$ matrix become impractical at this scale.

Each block in the figure maps well to a GPU on its own. The matrix multiplications are embarrassingly parallel: each output entry can be computed independently. The softmax requires a reduction along each row, but the rows themselves are independent and parallelize across N. The bottleneck is therefore not a lack of parallelism but the volume of computation and data movement on an $N \times N$ matrix at long context lengths. Most entries of the softmax output are close to zero (highlighted as "sparse" in Figure 1), and that observation motivates the tiled and sparse approaches we explore in the rest of this report.

Sparsity

A long line of work has shown that the score matrix $\sigma(QK^T)$ produced by attention is approximately sparse: most of its entries are very small after the row-wise softmax, and the output of MatMul (5) is dominated by a small fraction of the columns. This has motivated a broad family of *sparse attention* schemes that fall into two camps. Static patterns (such as local windows, strided, or block-diagonal masks) fix a sparsity structure ahead of time and are easy to map onto regular GPU tile layouts. Dynamic patterns (such as top-k or learned routing) choose which entries to keep based on the input itself, often giving better accuracy at a given sparsity level at the cost of more irregular computation [1].

Rather than commit to a single pattern, our experiments use a randomly-generated sparse mask. Each cell is independently kept with probability p, where p is the fraction of dense entries. This lets us sweep sparsity levels without entangling our results with the biases of any particular method. To match the granularity at which real-world transformers exploit sparsity, the mask is defined at a *block* level rather than per-element: a granularity of g means we partition the $N \times N$ score matrix into $g \times g$ tiles and label each tile as fully dense or fully sparse. A granularity of 1 corresponds to per-element sparsity, while a granularity of 32 means 32×32 tiles are the smallest unit of sparsity.

Block-level sparsity reduces the total amount of work on the score matrix, but it gives up structure that dense kernels rely on. Coalesced memory accesses, vectorized loads, and predictable load balancing all assume regular tile traversal; once tiles are skipped according to a mask, warps within a thread block can be assigned very different amounts of work, the rows of the score matrix lose alignment in memory, and arithmetic intensity drops. Recovering performance therefore requires both a sparse-friendly storage format and kernels that revisit the three stages of attention with the mask in mind.

BCSR storage. We store the block-level mask in *Blocked Compressed Sparse Row* (BCSR) format. BCSR is the block analogue of *Compressed Sparse Row* (CSR), a standard sparse-matrix layout that stores only the nonzero values along with the column index of each one and a per-row offset into that list. BCSR does the same thing at the granularity of fixed-size tiles: instead of recording the position of every individual nonzero, it records the position of every nonzero *tile*. For each row of tiles, we store the column indices of the dense tiles, and the dense tiles themselves are laid out contiguously in memory in row-major order. This keeps the inner loop of every kernel operating on dense, contiguous tiles (so coalescing and vectorization still apply within a tile) while cutting work in proportion to the fraction of sparse tiles.

Sparsity in MatMul. The two matrix multiplications in Figure 1 change shape under block sparsity. MatMul (4) computes QK^T, but only at positions where the mask is dense. This is the *sampled dense-dense matrix multiplication* (SDDMM) pattern: Q and K are dense, and the mask selects which output tiles to compute. Skipping a sparse output tile skips an entire block of dot-product reductions, which is where most of the savings come from. MatMul (5) computes $\sigma(QK^T) \cdot V$, where the left operand is now a block-sparse matrix in BCSR form and V is dense. This is the *sparse-dense matrix multiplication* (SpMM) pattern: each dense tile of the score matrix contributes a small dense matmul with the corresponding rows of V, and the BCSR row indexing tells each thread block which tiles to accumulate.

Sparsity in Softmax. The row-wise softmax sits between MatMul (4) and MatMul (5), and it has to respect the same mask. Sparse tiles can be treated as containing $-\infty$ scores, so they contribute zero to both the per-row sum and the output. We never materialize those tiles in practice: the softmax kernel walks the BCSR row directly, computes the per-row max and the exponential sum across only the dense tiles in that row, and writes back the normalized values in place. The reduction structure is the same as in dense softmax, but the length of each reduction is now proportional to the number of dense tiles in the row rather than to N.

Workload characterization. At moderate granularities, all three sparse stages stay data-parallel at the tile level. SDDMM (MatMul (4)) and SpMM (MatMul (5)) decompose into independent per-tile dense matrix multiplications with no cross-tile dependencies, while the sparse softmax does a per-row reduction (max and exponential sum) over only the dense tiles in a row, and the rows themselves remain independent. Locality is strong inside a tile because operands are dense and contiguous in memory, and BCSR's row-stationary layout keeps each row's column indices and dense values close together, so a kernel sweeping a row sees a tight access pattern. SIMD execution is effective on the inner $g \times g$ matmul, where each warp runs a regular dense kernel.

This picture degrades sharply at small granularities ($g \in \{1, 2, 4\}$). At $g = 1$ the workload is essentially a scatter-gather, and even at $g = 2$ or 4 each tile is smaller than a warp, so the inner dense matmul cannot fill 32 SIMT lanes and tensor-core or vector instructions cannot reach their minimum operand sizes. A single warp ends up assigned to multiple adjacent tiles whose mask bits diverge, forcing serialized execution within the warp. BCSR's per-tile metadata (a column-index lookup and a row-pointer dereference) also becomes comparable in cost to the actual compute, so memory traffic is dominated by indexing rather than values. Even at larger granularities the scheduling unit is now a variable-length list of dense tiles rather than a fixed row of N elements, so load imbalance across thread blocks is the primary cost we have to contain.

Approach

The main development focus of the project was on the three functions: Sparse Matrix Multiplication (SpMM), Sparse Dense Dense Matrix Multiplication (SDDMM), and Sparse Softmax. The entirety of the attention layer was written in CUDA, targeting an architecture of an RTX Blackwell 6000, which has a 128 KB L1 cache for shared memory, and 1024 threads in each thread block.

Tiled MatMul

Before describing the sparse functions, we first describe our hand written baseline, Tiled Matrix Multiplication. Given $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$, we compute $C = A \cdot B$.

The naive mapping launches one thread per output entry: thread (i, j) computes $C_{ij} = \sum_{k=0}^{K-1} A_{ik}B_{kj}$. Every thread streams a full row of A and a full column of B from global memory, even though threads in the same row of the output tile share the row of A and threads in the same column share the column of B. The reuse is large, but the naive kernel does not capture it.

The tiled version stages both operands through shared memory. We partition C into $T \times T$ tiles and assign one thread block of $T \times T$ threads to each tile, so every thread owns a single output entry. The block walks the shared K dimension in chunks of T: each iteration cooperatively loads a $T \times T$ stripe of A (rows $i_0{:}i_0 + T$, columns $k_0{:}k_0 + T$) and a $T \times T$ stripe of B (rows $k_0{:}k_0 + T$, columns $j_0{:}j_0 + T$) into shared buffers sA and sB, synchronizes, then has every thread accumulate its C entry as a dot product across the T columns of sA and rows of sB. The accumulator lives in a register across the entire K sweep and is written back once at the end. Each loaded entry of A is reused T times across the output tile's columns, and each entry of B is reused T times across its rows, which keeps the inner loop bandwidth-light. In practice we batch several T-wide stripes of K per iteration to amortize the cost of the per-chunk synchronization.

Algorithm 1 Tiled matrix multiplication for $C = A \cdot B$. One thread block per $T \times T$ output tile, one thread per output entry.

Block-level state: tile origin (i_0, j_0) in C; shared buffers $sA, sB \in \mathbb{R}^{T \times T}$
Thread-level state: local coords $(i, j) \in [0, T)^2$; register acc $\leftarrow 0$
for $k_0 = 0, T, 2T, \ldots, K - T$ **do**
  $sA[i, j] \leftarrow A[i_0 + i,\ k_0 + j]$
  $sB[i, j] \leftarrow B[k_0 + i,\ j_0 + j]$
  SYNCTHREADS
  for $k = 0$ **to** $T - 1$ **do**
    acc $\leftarrow$ acc $+ sA[i, k] \cdot sB[k, j]$
  end for
  SYNCTHREADS
end for
$C[i_0 + i,\ j_0 + j] \leftarrow$ acc

Sparse Matrix Multiplication (SpMM)

In SpMM we compute $C = AB$ where A is block-sparse (in BCSR form) and B is dense. The opportunity is that $C_{ij} = \sum_{k=0}^{K-1} A_{ik}B_{kj}$ only takes contributions from values of k where A_{ik} is dense, so most of the K-dimension reduction can be skipped.

General approach. The natural starting point is to graft the sparse skip onto tiled matmul. A thread block still owns a $T \times T$ output tile and walks the K dimension in T-wide chunks, but it only visits chunks that could contribute to its tile. The trouble is that a single block covers T different rows of A, and those rows may have different sparsity patterns, so the block's column schedule has to be the *union*

$$\mathcal{K} = \bigcup_{i \in [i_0, i_0 + T)} \{\, k : A_{ik} \text{ is dense} \,\},$$

i.e., every column index that is dense in at least one of the T rows. The block loads the dense entries of A on rows $[i_0, i_0 + T)$ together with the rows of B indexed by $k \in \mathcal{K}$ into shared memory, and each thread then accumulates over only the columns that are dense in *its* own row.

Algorithm 2 Sparse–dense matmul with row-union column scheduling. One thread block per $T \times T$ tile of C.

Block state: tile origin (i_0, j_0); $\mathcal{K} \leftarrow \bigcup_{i \in [i_0, i_0+T)} \{k : A_{ik} \text{ dense}\}$
Thread state: local coords $(i, j) \in [0, T)^2$; register acc $\leftarrow 0$
for each T-sized chunk of $\mathcal{K}$ **do**
  cooperatively load the dense $A_{i_0+i,\,k}$ for k in this chunk into sA
  cooperatively load $B_{k,\,j_0+j}$ for k in this chunk into sB
  SYNCTHREADS
  for each k in this chunk **do**
    if $A_{i_0+i,\,k}$ is dense **then**
      acc $\leftarrow$ acc $+ sA[i, k] \cdot sB[k, j]$
    end if
  end for
  SYNCTHREADS
end for
$C[i_0 + i,\, j_0 + j] \leftarrow$ acc

This formulation has two problems:

- **Load imbalance.** Threads in the same block iterate over different per-row sparsity patterns. The densest row sets the pace for the whole block, since every thread has to wait at SYNCTHREADS before the block can advance to the next chunk.
- **Wasted bandwidth.** The loaded sB chunk covers every column in $\mathcal{K}$, but each individual row of A uses only its own subset. Threads load entries of B they will never multiply, and the contiguous-column reads that vectorized loads depend on are punctured by skips wherever a row's pattern disagrees with $\mathcal{K}$.

Both problems come from forcing T rows with potentially different sparsity patterns to share a single column schedule. The cleanest fix is to stop combining rows.

Optimization 1: one row at a time. We change the block's footprint from a $T \times T$ tile of C to a $1 \times T^2$ stripe: a single row of A, paired with T^2 output columns. The thread count per block is unchanged, but now only one row of A is in play, so $\mathcal{K}$ collapses to the sparsity pattern of that single row. The union disappears, every thread that loads a column of B uses it, and every thread performs the same number of multiply-adds. Load imbalance and the vectorization problem both go away.

Optimization 2: align rows with the sparsity granularity. After implementing and testing the algorithm above, our measured speedups over the baseline were minimal. Profiling with Nsight Compute showed unexpectedly large load latencies in the kernel, and on inspection the cause was clear. Take $T = 16$ as a concrete case. In tiled matmul, a thread block outputs $T \times T = 256$ entries while loading $A_{i:i+T,\,k}$ and $B_{k,\,j:j+T}$ for all k, totaling $2TK$ elements. In the 1-row mapping above, the same block still outputs $T^2 = 256$ entries (now as a $1 \times T^2$ stripe), but loads $A_{i,\,k}$ and $B_{k,\,j:j+T^2}$ for all k, totaling $(T^2 + 1)K$ elements. That is roughly $16\times$ more memory traffic per output entry than the dense baseline, which matches the load times we saw in profiling.

We collapsed to a single row of A in Optimization 1 to guarantee a consistent sparsity pattern across the rows in a block. But that constraint is softer than it looks: by construction, the mask

is block-sparse at granularity G, so any consecutive G rows of A that fall inside the same row of $G \times G$ tiles share a sparsity pattern by definition. We therefore widen the block back out, mapping each block to a $G \times (T^2/G)$ tile of C. The total load per block becomes $(G + T^2/G)\,K$, which interpolates between the 1-row case at $G = 1$ and the dense baseline at $G = T$, while preserving the single-pattern invariant that made Optimization 1 work in the first place. This

Sparse Dense Dense Matrix Multiplication (SDDMM)

SDDMM computes $C = (AB) \odot M$, where A and B are dense and M is the sparse mask (in BCSR form). Only the entries of C at dense positions of M need to be produced, and the result is itself stored in BCSR. We tried three approaches; the third is the one we ended up using.

Naive: tiled matmul with a sparse output mask. The most direct approach is to keep the dense tiled matmul structure and skip writes for masked-out positions. Each thread block computes a $T \times T$ tile of the output exactly as in dense matmul, but threads whose output position is sparse simply return without writing. The dense loads of sA and sB are unchanged, so cache locality and vectorization are preserved. The cost is wasted computation: at sparsity above ~ 0.9, the vast majority of threads perform dot products whose results are never written. Workload imbalance also reappears, because the useful work per block depends on how many of its T^2 positions are dense, and at high sparsity an entire $T \times T$ tile can be launched with almost nothing to compute.

Row-batched dense Matrix Multiplication (`sddmm.cu`). Our first gather-based approach gathers at the BCSR block-row level. Each thread block is assigned one block-row b_i of C together with a contiguous *batch* of dense column-blocks within that row, where the batch size is a per-granularity constant tuned to fill the warps and fit in shared memory. The block first reads its batch's column indices from the BCSR `col_idx` array; then, for each 32-wide chunk of the K dimension, it loads (i) the $T \times 32$ stripe of A corresponding to its block-row, once, and (ii) the $T \times 32$ stripe of B for each column-block in the batch, gathered into one shared-memory buffer. The inner loop is then a clean dense matmul: every thread accumulates a 32-element dot product per K-chunk with no mask checks, because the column-blocks were filtered to the dense ones during the gather. After the K sweep, the accumulators are written directly into C's BCSR storage. The cost of loading A amortizes across every column-block in the batch, and the mask is consulted once per batch rather than once per output entry.

The drawback is grid sizing: different block-rows have different numbers of dense column-blocks, so the grid is sized for the *worst* row. Block-rows with fewer dense blocks have leftover thread blocks that exit immediately, and at low granularity the per-row variance is large enough that this launch overhead grows noticeable.

Active-coordinate gather. The version we ended up using inverts the order of operations: first gather the positions that need to be computed, then compute only those. The output is partitioned into 32×32 SDDMM tiles, which sit on top of the smaller $T \times T$ BCSR blocks of the mask (with $T \leq 32$). One warp owns each SDDMM tile and processes it in three stages.

1. **Dynamic tile scheduling.** Warps pull the next 32×32 tile via an atomic increment of a global counter, rather than from a static grid. Warps that finish quickly grab another tile, and warps that land on a fully-sparse tile move on after almost no work.

2. **Active-coordinate list.** The warp walks the BCSR row-pointers of M to find the dense $T \times T$ blocks that intersect its 32×32 tile, and appends the (r, c) coordinates of every entry inside those blocks to a per-warp list in shared memory. If the list is empty after this pass, the warp is done with the tile.

3. **Dense load, sparse compute, sparse write-back.** The warp iterates the N dimension in 32-wide chunks. Each chunk loads the full 32×32 stripes of A and B into shared memory with regular coalesced reads, then computes a dot product *only* at the coordinates in the active list. After all N-chunks finish, the accumulators are written back into the BCSR storage of C at exactly those positions.

This recovers the dense-load patterns of tiled matmul (the A and B tiles are read densely and contiguously, so coalescing and vectorization still apply) while avoiding the wasted arithmetic of the naive variant: every multiply-add lands at a position whose result will actually be stored. The dynamic counter also keeps warps busy regardless of how the dense tiles are distributed in C, so the kernel behaves consistently as G varies.

SoftMax

TODO TODO TODO TODO

Results

We measure performance by running one full attention iteration end-to-end and comparing our sparse implementation's wall-clock time against a dense baseline. The baseline runs the same attention computation, but uses our optimized tiled-matmul kernel for both QK^T and the multiplication with V, and a standard dense row-wise softmax in between. It does not exploit the mask in any way: it materializes the full $N \times N$ score matrix and processes every entry. Both implementations are written in CUDA, target the same RTX Blackwell 6000, and are timed end-to-end as well as kernel-by-kernel.

We sweep three hyperparameters:

- N, the context length (the problem size), $\in \{2^{10}, 2^{12}, 2^{14}, 2^{15}\}$.
- p, the fraction of sparse entries in the mask, $\in \{0.5, 0.9, 0.95, 0.99\}$.
- G, the sparsity granularity, $\in \{1, 2, 8, 32\}$.

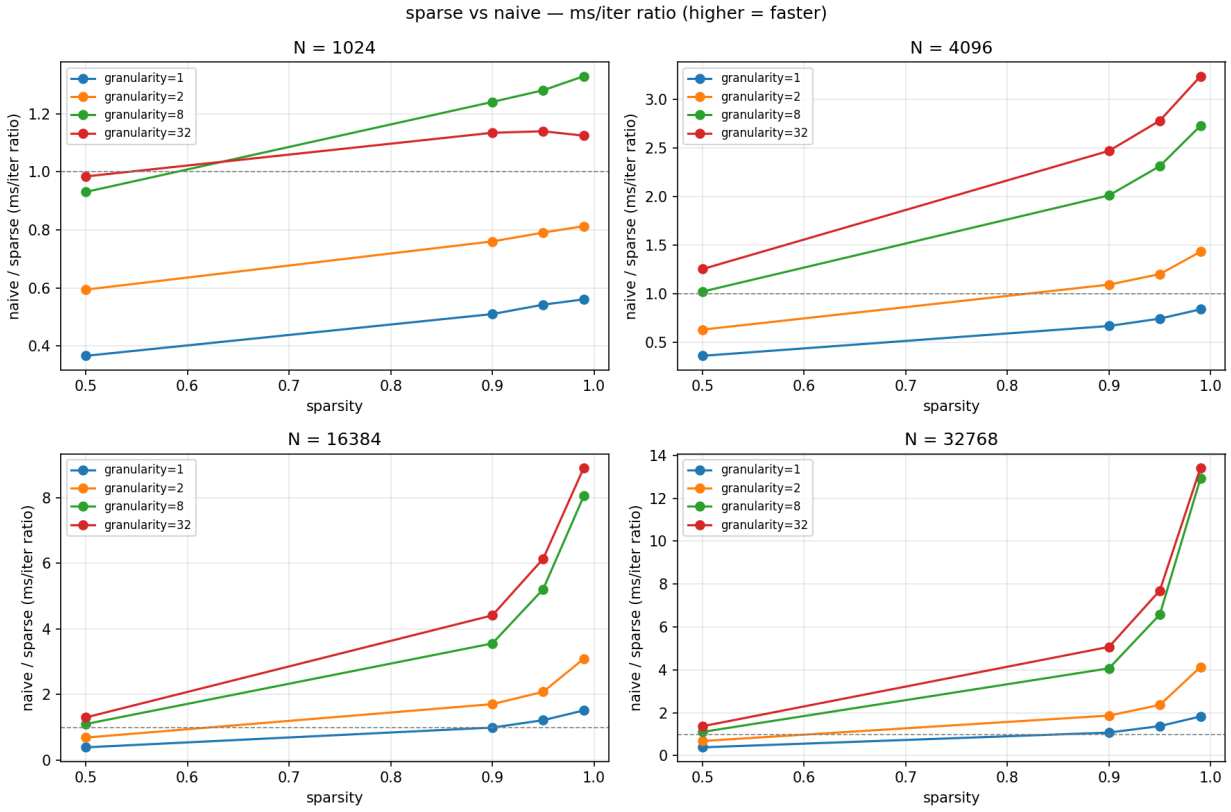
Experimental setup. For each (N, p, G) configuration we run both the sparse and the dense baseline on three random seeds. The seed controls the input matrices and the sparsity pattern, so both implementations see identical inputs and an identical mask on each run. We record per-kernel wall-clock times alongside the end-to-end attention time, and report the mean across the three seeds.

Hyperparameter notes. Two observations frame the rest of this section:

- Increasing p reduces the amount of useful arithmetic. An ideally-parallel sparse implementation would therefore reach a speedup of $1/(1-p)$ over the dense baseline ($100\times$ at $p = 0.99$). That ratio is an upper bound; sequential portions and per-tile overhead always pull the realized speedup below it.

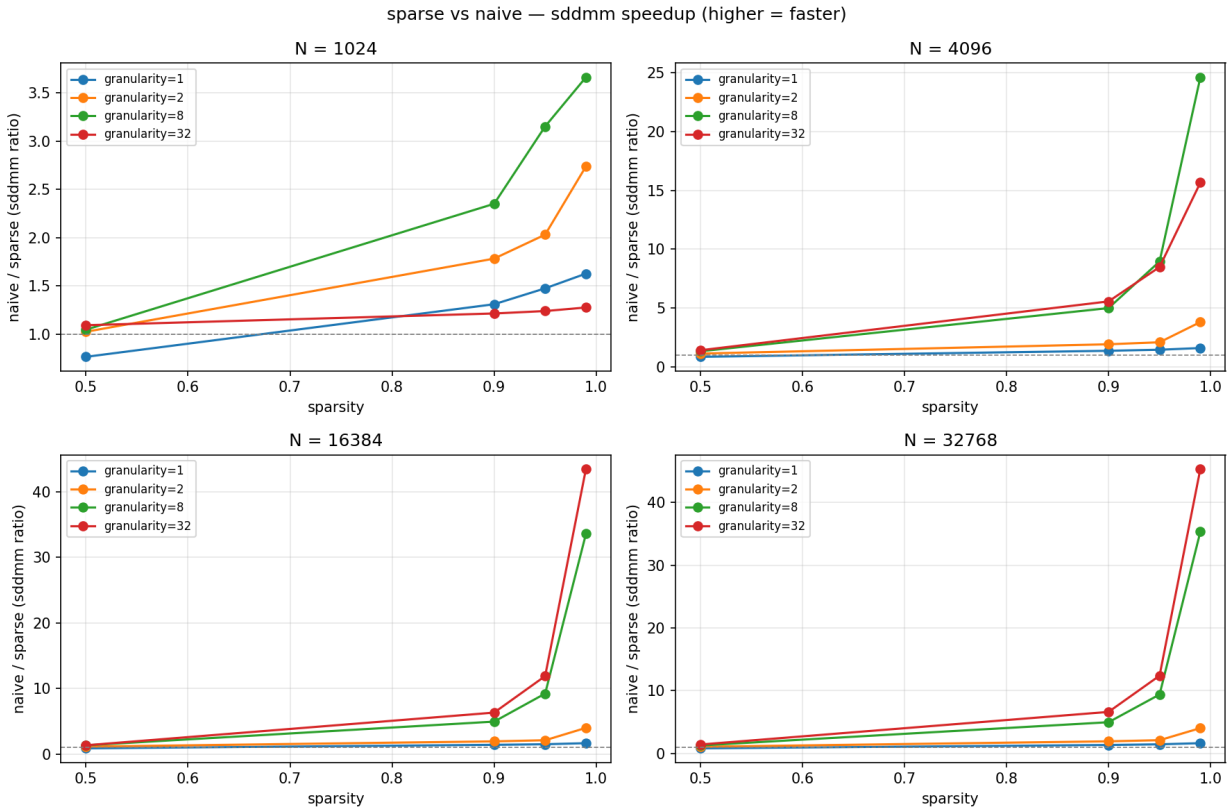
- Increasing G makes the sparsity pattern coarser and less expressive: at $G = 32$, every entry inside a 32×32 tile shares the same dense/sparse decision. Coarser patterns lose modeling fidelity, but they also let the kernels exploit larger contiguous regions, recovering vectorized loads and warp-aligned compute. The sweep over G exists to surface this tradeoff; in a production setting the granularity would be chosen empirically for the workload rather than from a uniform-random mask.

General Speedup

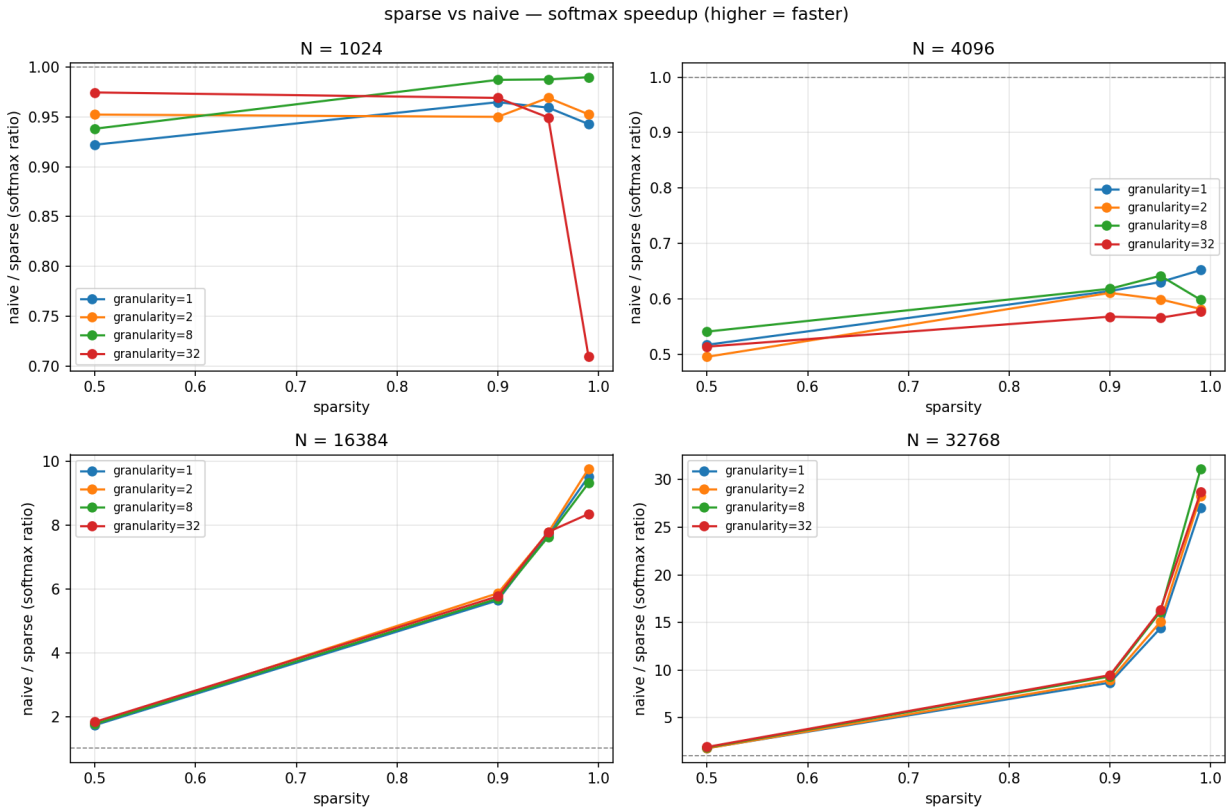


Looking at the speedup graphs, it's clear that problem size has a strong correlation with speedup.

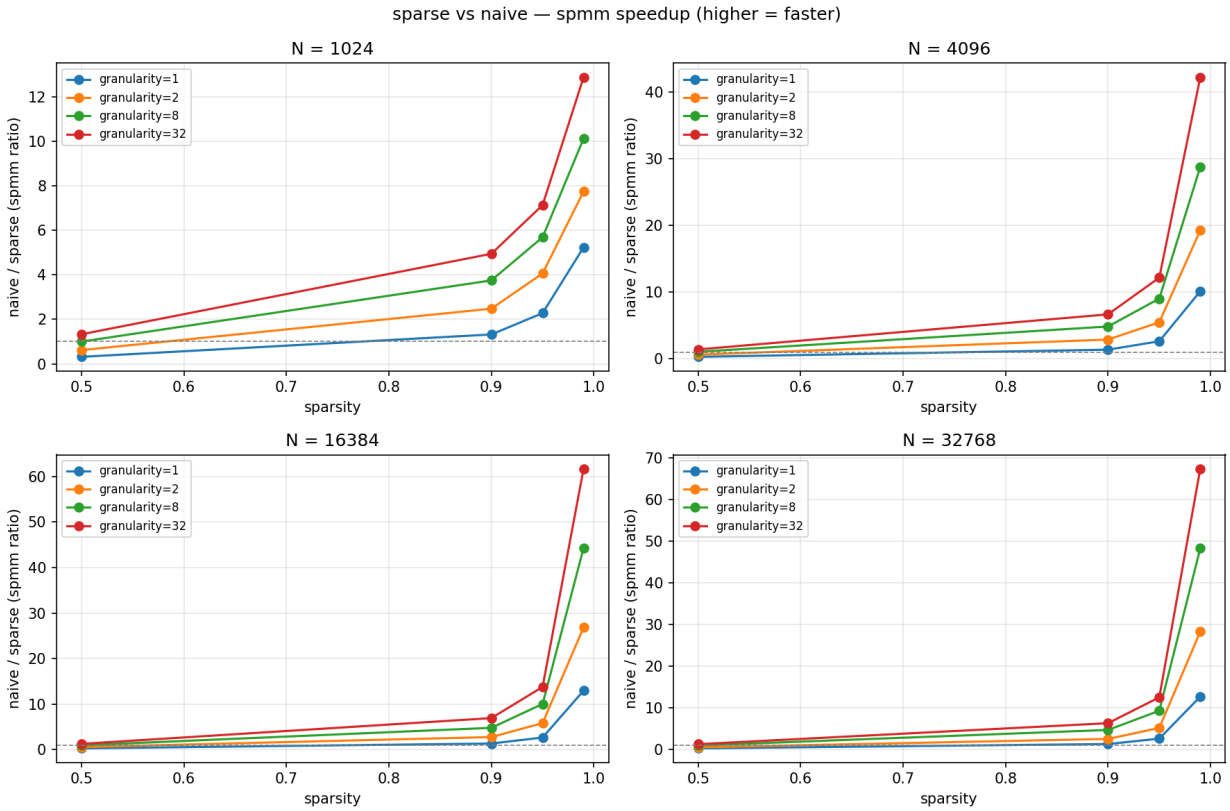
SDDMM


sparse vs naive — sddmm speedup (higher = faster)
N = 1024
N = 4096
N = 16384
N = 32768
granularity=1
granularity=2
granularity=8
granularity=32
naive / sparse (sddmm ratio)
sparsity

SoftMax


sparse vs naive — softmax speedup (higher = faster)
N = 1024
N = 4096
N = 16384
N = 32768
naive / sparse (softmax ratio)
sparsity
granularity=1
granularity=2
granularity=8
granularity=32

SpMM



References

[1] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers, 2021.

List of work

The total credit for the project was 50-50 between Saransh and Mehul.